Programski prevodioci

08 Generisanje koda - micko

25-26

Dunja Vrbaški

Fakultet tehničkih nauka, Novi Sad

```
output = fopen("output.asm", "w+");
```

micko.y

```
#define code(args...) ({fprintf(output, args); \
          if (++out_lin > 2000) err("Too many output lines"), exit(1); })
```

defs.h

```
#define LAST_WORKING_REG      12
#define FUN_REG               13
```

defs.h

```
int free_reg_num = 0;
```

codegen.c

```
int  take_reg(void);

void free_reg(void);

void free_if_reg(int reg_index);

void gen_sym_name(int index);

void gen_cmp(int op1_index, int op2_index);

void gen_mov(int input_index, int output_index);
```

```
// zauzimanje registra (prvog slobodnog)
int  take_reg(void); //uslov + free_reg_num++


// oslobadjanje registra (poslednjeg zauzetog)
void free_reg(void); //uslov + free_reg_num–


// oslobadjanje registra za zadati indeks
void free_if_reg(int);
```

- radni registri: 0-12
- zauzimanje: stek
- svako čitanje - oslobađanje
- tabela simbola - tip vrednosti

```c
int take_reg(void) {
  if(free_reg_num > LAST_WORKING_REG) {
    err("Compiler error! No free registers!");
    exit(EXIT_FAILURE);
  }
  return free_reg_num++;
}
```

```c
void free_reg(void) {
   if(free_reg_num < 1) {
      err("Compiler error! No more registers to free!");
      exit(EXIT_FAILURE);
   }
   else
      set_type(--free_reg_num, NO_TYPE);
}
```

```c
void free_if_reg(int reg_index) {
  if(reg_index >= 0 && reg_index <= LAST_WORKING_REG)
    free_reg();
}
```

```
// GK na osnovu vrste simbola
void gen_sym_name(int index) {
  if(index > -1) {
    if(... == VAR)
      ...-%d(%%14)...
    else
       if(... == PAR)
        ...%d(%%14)...
       else
        if(... == LIT)
          ...$%s...
        else // function, reg
          ...%s...
  }
}
```

```
void gen_sym_name(int index) {
  if(index > -1) {
    if(get_kind(index) == VAR) // -n*4(%14)
      code("-%d(%%14)", get_atr1(index) * 4);
    else
      if(get_kind(index) == PAR) // m*4(%14)
        code("%d(%%14)", 4 + get_atr1(index) *4);
      else
        if(get_kind(index) == LIT)
          code("$%s", get_name(index));
        else //function, reg
          code("%s", get_name(index));
  }
}
```

```
void gen_cmp(...op1..., ...op2...) {
  if(...op type... == INT)
    ...CMPS...
  else
    ...CMPU...

  gen_sym_name op1...
  “,”
  gen_sym_name op2...
  ...
}
```

```
void gen_cmp(int op1_index, int op2_index) {
  if(get_type(op1_index) == INT)
    code("\n\t\tCMPS \t");
  else
    code("\n\t\tCMPU \t");
  gen_sym_name(op1_index);
  code(",");
  gen_sym_name(op2_index);
  free_if_reg(op2_index);
  free_if_reg(op1_index);
}
```

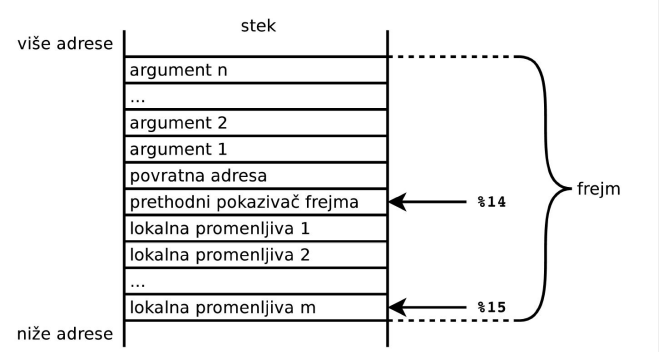
```
void gen_mov(...input..., ...output...) {
  ...MOV...

  gen_sym_name input...
  “,”
  gen_sym_name output...
  ...
}
```

```c
void gen_mov(int input_index, int output_index) {
  code("\n\t\tMOV \t");
  gen_sym_name(input_index);
  code(",");
  gen_sym_name(output_index);

  //ako se smešta u registar, treba preneti tip
  if(output_index >= 0 && output_index <= LAST_WORKING_REG)
    set_type(output_index, get_type(input_index));

  free_if_reg(input_index);
}
```

Funkcija

```c
int f(int p) {
   int a;
   return p + a;
}
```

```
f:                                  1
   PUSH    %14                      2
   MOV     %15,%14                  3
   SUBS    %15,$4,%15               4
@f_body:                            5
   ADDS    8(%14),-4(%14),%0        6
   MOV     %0,%13                   7
   JMP     @f_exit                  8
@f_exit:                            9
   MOV     %14,%15                  10
   POP     %14                      11
   RET                              12
```



$13

$14

$15

```
function
  : type _ID
      {
         fun_idx = lookup_symbol($2, FUN);
         if(fun_idx == NO_INDEX)
            fun_idx = insert_symbol($2,FUN,$1,NO_ATR,NO_ATR);
         else
            err("redefinition of function '%s'", $2);
      }
    _LPAREN parameter _RPAREN body
      {
         clear_symbols(fun_idx + 1);
         var_num = 0;
      }
  ;
```

GK?

```
function
  : type _ID
      {
         fun_idx = lookup_symbol($2, FUN);
         if(fun_idx == NO_INDEX)
            fun_idx = insert_symbol($2,FUN,$1,NO_ATR,NO_ATR);
         else
            err("redefinition of function '%s'", $2);
      }
    _LPAREN parameter _RPAREN body
      {
         clear_symbols(fun_idx + 1);
         var_num = 0;
      }
  ;
```

```
f:
   PUSH  %14
   MOV   %15,%14

- generiši labelu
- upiši prethodni fp
- postavi novi fp
```

```
   SUBS  %15,$4,%15
@f_body:
   ...
   MOV   %0,%13
   JMP   @f_exit

-   zauzmi za lokalne
-   generiši labelu
-   ...
-   vrati vrednost
-   generiši skok na labelu
```

```
@f_exit:
   MOV   %14,%15
   POP   %14
   RET

- generiši labelu
- izbriši lokalne (pomeri sp)
- vrati stari fp
- postavi prog.br.
```

```
function
  : type _ID
      {
         fun_idx = lookup_symbol($2, FUN);
         if(fun_idx == NO_INDEX)
            fun_idx = insert_symbol($2,FUN,$1,NO_ATR,NO_ATR);
         else
            err("redefinition of function '%s'", $2);

        code("\n%s:", $2);
        code("\n\t\t\tPUSH\t%%14");
        code("\n\t\t\tMOV \t%%15,%%14");

      }
    _LPAREN parameter _RPAREN body
      {
        // u nastavku
      }

  ;
```

```
f:
   PUSH  %14
   MOV   %15,%14
```

- generiši labelu
- upiši prethodni fp
- postavi novi fp

```
function
  : type _ID
      {
         // prethodno
      }
    _LPAREN parameter _RPAREN body
      {
         clear_symbols(fun_idx + 1);
         var_num = 0;

         code("\n@%s_exit:", $2);
         code("\n\t\t\tMOV \t%%14,%%15");
         code("\n\t\t\tPOP \t%%14");
         code("\n\t\t\tRET");
      }
  ;
```

```
@f_exit:
   MOV   %14,%15
   POP   %14
   RET
```

- generiši labelu
- izbriši lokalne (pomeri sp)
- vrati stari fp
- postavi prog.br.

Telo funkcije

```
body  : _LBRACKET variable_list statement_list _RBRACKET  ;
```

Zauzmi za LP, generiši labelu za body

Generiše se ostatak, između ostalog i povratna vrednost kroz return_statement

```
body
  : _LBRACKET variable_list
      {
         if(var_num)
            code("\n\t\t\tSUBS\t%%15,$%d,%%15", 4*var_num);
         code("\n@%s_body:", get_name(fun_idx));
      }
    statement_list _RBRACKET
  ;
```

```
   SUBS  %15,$4,%15
@f_body:
   ...
   MOV   %0,%13
   JMP   @f_exit
```

- zauzmi za lokalne
- generiši labelu
- ...
- vrati vrednost
- generiši skok na labelu

Povratna vrednost

```
return_statement
  : _RETURN num_exp _SEMICOLON
      {
        if(get_type(fun_idx) != get_type($2))
          err("incompatible types in return");
      }
  ;
```

Generiši povratnu vrednost
Generiši skok na exit

```
return_statement
  : _RETURN num_exp _SEMICOLON
      {
        if(get_type(fun_idx) != get_type($2))
          err("incompatible types in return");

        gen_mov($2, FUN_REG);
        code("\n\t\t\tJMP \t@%s_exit", get_name(fun_idx));
      }
  ;
```

```
    SUBS   %15,$4,%15
@f_body:
    ...
    MOV    %0,%13
    JMP    @f_exit

-   zauzmi za lokalne
-   generiši labelu
-   ...
-   vrati vrednost
-   generiši skok na labelu
```

```
function
  : type _ID
      {
         fun_idx = lookup_symbol($2, FUN);
         if(fun_idx == NO_INDEX)
            fun_idx = insert_symbol($2,FUN,$1,NO_ATR,NO_ATR);
         else
            err("redefinition of function '%s'", $2);
      }
    _LPAREN parameter _RPAREN body
      {
         clear_symbols(fun_idx + 1);
         var_num = 0;
      }
  ;
```

```
f:
   PUSH  %14
   MOV   %15,%14

- generiši labelu
- upiši prethodni fp
- postavi novi fp
```

```
   SUBS  %15,$4,%15
@f_body:
   ...
   MOV   %0,%13
   JMP   @f_exit

-   zauzmi za lokalne
-   generiši labelu
-   ...
-   vrati vrednost
-   generiši skok na labelu
```

```
@f_exit:
   MOV   %14,%15
   POP   %14
   RET

- generiši labelu
- izbriši lokalne (pomeri sp)
- vrati stari fp
- postavi prog.br.
```

Poziv funkcije

```
a = f(a + b);
```

```
ADDS    -4(%14),-8(%14),%0    1
PUSH    %0                    2
CALL    f                     3
ADDU    %15,$4,%15            4
MOV     %13,%0                5
MOV     %0,-4(%14)            5
```

- GK za preuzimanje vrednosti iz %13 nema veze sa pozivom funkcije.
- Kad dodela bude generisala kod – generisaće se upravo MOV sa tim registrom.
- Semantička vrednost se svakako postavlja na indeks u TS za registar %13.

```
function_call
   : _ID
      {
         fcall_idx = lookup_symbol($1, FUN);
         if(fcall_idx == NO_INDEX)
            err("'%s' is not a function", $1);
      }
    _LPAREN argument _RPAREN
      {
         if(get_atr1(fcall_idx) != $4)
            err("wrong number of arguments");

         set_type(FUN_REG, get_type(fcall_idx));
         $$ = FUN_REG;
      }
   ;
```

```
ADDS   -4(%14),-8(%14),%0
PUSH   %0
```

```
CALL   f
ADDU   %15,$4,%15
```

```
MOV    %13,%0
MOV    %0,-4(%14)
```

```
function_call
   : _ID
      {
         fcall_idx = lookup_symbol($1, FUN);
         if(fcall_idx == NO_INDEX)
            err("'%s' is not a function", $1);
      }
    _LPAREN argument _RPAREN
      {
         if(get_atr1(fcall_idx) != $4)
            err("wrong number of arguments");

         code("\n\t\t\tCALL\t%s", get_name(fcall_idx));
         if($4 > 0)
            code("\n\t\t\tADDS\t%%15,$%d,%%15", $4 * 4);


         set_type(FUN_REG, get_type(fcall_idx));
         $$ = FUN_REG;
      }
   ;
```

```
CALL   f
ADDU   %15,$4,%15
```

Argument

```
a = f(a + b);
```

```
ADDS    -4(%14),-8(%14),%0    1
PUSH    %0                    2
CALL    f                     3
ADDU    %15,$4,%15            4
MOV     %13,-4(%14)           5
```

```
argument
  : /* empty */
    { $$ = 0; }
  | num_exp
    {
       if(get_atr2(fcall_idx) != get_type($1))
        err("incompatible type for argument in '%s'",
            get_name(fcall_idx));
      $$ = 1;
    }
  ;
```

```
ADDS  -4(%14),-8(%14),%0
```

- generiši vrednost argumenta
- ne mora biti ovaj izraz

```
PUSH $0
```

- generiši postavljanje na stek
- ne mora biti sadržaj %0

```
argument
  : /* empty */
    { $$ = 0; }
  | num_exp
    {
       if(get_atr2(fcall_idx) != get_type($1))
        err("incompatible type for argument in '%s'",
            get_name(fcall_idx));

      free_if_reg($1);
      code("\n\t\t\tPUSH\t");
      gen_sym_name($1);

      $$ = 1;
    }
  ;
```

```
ADDS  -4(%14),-8(%14),%0
```

- generiši vrednost argumenta
- ne mora biti ovaj izraz

```
PUSH $0
```

- generiši postavljanje na stek
- ne mora biti sadržaj %0

if-else

```
if(a < b)
   a = 1;
else
   a = 2;
```

```
@if0:
   CMPS  -4(%14),-8(%14)        1
   JGES  @false0                2
@true0:                         3
   MOV   $1,-4(%14)             4
   JMP   @exit0                 5
@false0    :                    6
   MOV   $2,-4(%14)             7
@exit0:                         8
```

```
if_statement
  : if_part %prec ONLY_IF
  | if_part _ELSE statement
  ;

if_part
  : _IF _LPAREN
    rel_exp
    _RPAREN
    statement
  ;
```

```
@if0:
```

- generiši labelu za početak

```
   CMPS  -4(%14),-8(%14)
```

- generiši uslov

```
   JGES  @false0
@true0:
```

- generiši skok na false
- generiši labelu za true

```
   MOV   $1,-4(%14)
```

- generiši telo za if

```
   JMP   @exit0
@false0:
```

- generiši skok na kraj
- generiši labelu za false

```
   MOV   $2,-4(%14)
```

- generiši telo za else

```
@exit0:
```

- generiši labelu za izlaz

```
if_statement
  : if_part %prec ONLY_IF
  | if_part _ELSE statement
  ;

if_part
  : _IF _LPAREN
    rel_exp
    _RPAREN
    statement
  ;
```

```
@if0:

- generiši labelu za početak
```

```
   CMPS  -4(%14),-8(%14)

- generiši uslov
```

```
   JGES  @false0
@true0:

- generiši skok na false
- generiši labelu za true
```

```
   MOV   $1,-4(%14)

- generiši telo za if
```

```
   JMP   @exit0
@false0:

- generiši skok na kraj
- generiši labelu za false
```

```
   MOV   $2,-4(%14)

- generiši telo za else
```

```
@exit0:

- generiši labelu za izlaz
```

```
if_statement
  : if_part %prec ONLY_IF
  | if_part _ELSE statement
  ;

if_part
  : _IF _LPAREN
    rel_exp
    _RPAREN
    statement
  ;
```

```
@if0:

- generiši labelu za početak
```

```
   CMPS  -4(%14),-8(%14)

- generiši uslov
```

```
   JGES  @false0
@true0:

- generiši skok na false
- generiši labelu za true
```

```
   MOV   $1,-4(%14)

- generiši telo za if
```

```
   JMP   @exit0
@false0:

- generiši skok na kraj
- generiši labelu za false
```

```
   MOV   $2,-4(%14)

- generiši telo za else
```

```
@exit0:

- generiši labelu za izlaz
```

```
if_statement
  : if_part %prec ONLY_IF
  | if_part _ELSE statement
  ;

if_part
  : _IF _LPAREN
    rel_exp
    _RPAREN
    statement
  ;
```

```
@if0:

- generiši labelu za početak
```

```
   JGES  @false0
@true0:

- generiši skok na false
- generiši labelu za true
```

```
   JMP   @exit0
@false0:

- generiši skok na kraj
- generiši labelu za false
```

```
@exit0:

- generiši labelu za izlaz
```

```
if_statement
  : if_part %prec ONLY_IF
  | if_part _ELSE statement
  ;

if_part
  : _IF _LPAREN
      {
        $<i>$ = ++lab_num;
        code("\n@if%d:", lab_num);
      }
    rel_exp
      {
        ... JGES u nastavku...
        code("\n@true%d:", $<i>3);
      }
    _RPAREN statement
      {
        code("\n\t\tJMP \t@exit%d", $<i>3);
        code("\n@false%d:", $<i>3);
        $$ = $<i>3;
      }
  ;
```

```
@if0:

- generiši labelu za početak
```

```
   JGES  @false0
@true0:

- generiši skok na false
- generiši labelu za true
```

```
   JMP   @exit0
@false0:

- generiši skok na kraj
- generiši labelu za false
```

if_part nije imao semantičke vrednosti u prethodnim verzijama

```
if_statement
  : if_part %prec ONLY_IF
  | if_part _ELSE statement
  ;

if_part
  : _IF _LPAREN
      {
        $<i>$ = ++lab_num;
        code("\n@if%d:", lab_num);
      }
    rel_exp
      {
        ... JGES u nastavku...
        code("\n@true%d:", $<i>3);
      }
    _RPAREN statement
      {
        code("\n\t\tJMP \t@exit%d", $<i>3);
        code("\n@false%d:", $<i>3);
        $$ = $<i>3;
      }
  ;
```

```
@if0:

- generiši labelu za početak
```

```
   JGES  @false0
@true0:

- generiši skok na false
- generiši labelu za true
```

```
   JMP   @exit0
@false0:

- generiši skok na kraj
- generiši labelu za false
```

```
if_statement
  : if_part %prec ONLY_IF
  | if_part _ELSE statement
  ;

if_part
  : _IF _LPAREN
      {
        $<i>$ = ++lab_num;
        code("\n@if%d:", lab_num);
      }
    rel_exp
      {
        code("\n\t\t%s\t@false%d", opp_jumps[$4], $<i>3);
        code("\n@true%d:", $<i>3);
      }
    _RPAREN statement
      {
        code("\n\t\tJMP \t@exit%d", $<i>3);
        code("\n@false%d:", $<i>3);
        $$ = $<i>3;
      }
  ;
```

```
@if0:

- generiši labelu za početak
```

```
   JGES  @false0
@true0:

- generiši skok na false
- generiši labelu za true
```

```
   JMP   @exit0
@false0:

- generiši skok na kraj
- generiši labelu za false
```

Semantička vrednost relacionog izraza će nam dati informaciju koji uslovni skok da koristimo

```
static char* opp_jumps[]={"JGES", "JLES", "JGTS", "JLTS", "JNE ", "JEQ ",
                          "JGEU", "JLEU", "JGTU", "JLTU", "JNE ", "JEQ "};
```

defs.h

⇒ Semantička vrednost relacionog izraza za naš primer (a < b) bi morala biti 0

Kasnije će biti prikazan rel_exp

```
if_statement
  : if_part %prec ONLY_IF
  | if_part _ELSE statement
  ;

if_part
  : _IF _LPAREN
      {
        $<i>$ = ++lab_num;
        code("\n@if%d:", lab_num);
      }
    rel_exp
      {
        code("\n\t\t%s\t@false%d", opp_jumps[$4], $<i>3);
        code("\n@true%d:", $<i>3);
      }
    _RPAREN statement
      {
        code("\n\t\tJMP \t@exit%d", $<i>3);
        code("\n@false%d:", $<i>3);
        $$ = $<i>3;
      }

  ;
```

```
@if0:

- generiši labelu za početak
```

```
   JGES  @false0
@true0:

- generiši skok na false
- generiši labelu za true
```

```
   JMP   @exit0
@false0:

- generiši skok na kraj
- generiši labelu za false
```

```
@exit0:

- generiši labelu za izlaz
```

```
if_statement
 : if_part %prec ONLY_IF
      { code("\n@exit%d:", $1); }
 | if_part _ELSE statement
      { code("\n@exit%d:", $1); }
 ;

if_part
  : _IF _LPAREN
      {
        $<i>$ = ++lab_num;
        code("\n@if%d:", lab_num);
      }
    rel_exp
      {
        code("\n\t\t%s\t@false%d", opp_jumps[$4], $<i>3);
        code("\n@true%d:", $<i>3);
      }
    _RPAREN statement
      {
        code("\n\t\tJMP \t@exit%d", $<i>3);
        code("\n@false%d:", $<i>3);
        $$ = $<i>3;
      }

  ;
```

```
@exit0:
```

- generiši labelu za izlaz

```
if_statement
  : if_part %prec ONLY_IF
  | if_part _ELSE statement
  ;

if_part
  : _IF _LPAREN
    rel_exp
    _RPAREN
    statement
  ;
```

```
@if0:

- generiši labelu za početak
```

```
   CMPS  -4(%14),-8(%14)

- generiši uslov
```

```
   JGES  @false0
@true0:

- generiši skok na false
- generiši labelu za true
```

```
   MOV   $1,-4(%14)

- generiši telo za if
```

```
   JMP   @exit0
@false0:

- generiši skok na kraj
- generiši labelu za false
```

```
   MOV   $2,-4(%14)

- generiši telo za else
```

```
@exit0:

- generiši labelu za izlaz
```

```
if_statement
  : if_part %prec ONLY_IF
  | if_part _ELSE statement
  ;

if_part
  : _IF _LPAREN
    rel_exp
    _RPAREN
    statement
  ;
```

```
@if0:

- generiši labelu za početak
```

```
   CMPS  -4(%14),-8(%14)

- generiši uslov
```

```
   JGES  @false0
@true0:

- generiši skok na false
- generiši labelu za true
```

```
   MOV   $1,-4(%14)

- generiši telo za if
```

```
   JMP   @exit0
@false0:

- generiši skok na kraj
- generiši labelu za false
```

```
   MOV   $2,-4(%14)

- generiši telo za else
```

```
@exit0:

- generiši labelu za izlaz
```

Relacioni izrazi

```
if(a < b)
   a = 1;
else
   a = 2;
```

```
@if0:
   CMPS  -4(%14),-8(%14)        1
   JGES  @false0                2
@true0:                         3
   MOV   $1,-4(%14)             4
   JMP   @exit0                 5
@false0    :                    6
   MOV   $2,-4(%14)             7
@exit0:                         8
```

```
rel_exp
  : num_exp _RELOP num_exp
      {
        if(get_type($1) != get_type($3))
          err("invalid operands: relational operator");
      }
  ;
```

```
    CMPS  -4(%14),-8(%14)
```

- postavi semantičku vrednost
- generiši uslov

```
rel_exp
  : num_exp _RELOP num_exp
      {
        if(get_type($1) != get_type($3))
          err("invalid operands: relational operator");

        ...postavi vrednost u nastavku...
        gen_cmp($1, $3);
      }
  ;
```

```
    CMPS   -4(%14),-8(%14)
```

- postavi semantičku vrednost
- generiši uslov

```
rel_exp
  : num_exp _RELOP num_exp
      {
        if(get_type($1) != get_type($3))
          err("invalid operands: relational operator");

        $$ = $2 + ((get_type($1) - 1) * RELOP_NUMBER);
        gen_cmp($1, $3);
      }
  ;
```

```
    CMPS   -4(%14),-8(%14)
```

- postavi semantičku vrednost
- generiši uslov

```
"<"                    { yylval.i = LT; return _RELOP; }
">"                    { yylval.i = GT; return _RELOP; }
"<="                   { yylval.i = LE; return _RELOP; }
">="                   { yylval.i = GE; return _RELOP; }
"=="                   { yylval.i = EQ; return _RELOP; }
"!="                   { yylval.i = NE; return _RELOP; }
```

```
enum relops { LT, GT, LE, GE, EQ, NE, RELOP_NUMBER };
```

```
enum types { NO_TYPE, INT, UINT };
```

```
if_statement
  : if_part %prec ONLY_IF
  | if_part _ELSE statement
  ;

if_part
  : _IF _LPAREN
      {
        $<i>$ = ++lab_num;
        code("\n@if%d:", lab_num);
      }
    rel_exp
      {
        code("\n\t\t%s\t@false%d", opp_jumps[$4], $<i>3);
        code("\n@true%d:", $<i>3);
      }
    _RPAREN statement
      {
        code("\n\t\tJMP \t@exit%d", $<i>3);
        code("\n@false%d:", $<i>3);
        $$ = $<i>3;
      }
  ;
```

```
   JGES  @false0
@true0:

- generiši skok na false
- generiši labelu za true
```

```
static char* opp_jumps[]={"JGES", "JLES", "JGTS", "JLTS", "JNE ", "JEQ ",
                          "JGEU", "JLEU", "JGTU", "JLTU", "JNE ", "JEQ "};
```

Skokovi su “obrnuti”, negacija:
a<b □ JGES

Numerički izraz

```
a = f(a + b);
```

```
ADDS  -4(%14),-8(%14),%0        1
PUSH  %0                        2
CALL f                          3
ADDU %15,$4,%15                 4
MOV %13,%0                      5
MOV %0,-4(%14)                  6
```

```
num_exp
  : exp
  | num_exp _AROP exp
      {
        if(get_type($1) != get_type($3))
          err("invalid operands: arithmetic operation");
      }
  ;
```

```
ADDS  -4(%14),-8(%14),%0

- generiši operaciju
- generiši ulazni operand 1
-  generiši ulazni operand 2
-  generiši izlazni operand
```

```
num_exp
  : exp
  | num_exp _AROP exp
      {
        if(get_type($1) != get_type($3))
          err("invalid operands: arithmetic operation");
      }

      int t1 = get_type($1);
      code("\n\t\t%s\t", ar_instructions[$2 + (t1 - 1) * AROP_NUMBER]);
      ...
  ;
```

```
ADDS   -4(%14),-8(%14),%0

- generiši operaciju
- generiši ulazni operand 1
-  generiši ulazni operand 2
-  generiši izlazni operand
```

```
"+"                    { yylval.i = ADD; return _AROP; }
"-"                    { yylval.i = SUB; return _AROP; }
```

```
enum types { NO_TYPE, INT, UINT };
```

```
enum arops { ADD, SUB, MUL, DIV, AROP_NUMBER };
```

```
static char *ar_instructions[] = { "ADDS", "SUBS", "MULS", "DIVS",
                                   "ADDU", "SUBU", "MULU", "DIVU" };
```

```
num_exp
  : exp
  | num_exp _AROP exp
      {
        if(get_type($1) != get_type($3))
          err("invalid operands: arithmetic operation");
      }
      int t1 = get_type($1);
      code("\n\t\t%s\t", ar_instructions[$2 + (t1 - 1) * AROP_NUMBER]);

      gen_sym_name($1);
      code(",");
      gen_sym_name($3);
      code(",");

      free_if_reg($3);
      free_if_reg($1);

      ...


  ;
```

```
ADDS   -4(%14),-8(%14),%0
```

- generiši operaciju
- generiši ulazni operand 1
- generiši ulazni operand 2
- generiši izlazni operand

```
num_exp
  : exp
  | num_exp _AROP exp
      {
        if(get_type($1) != get_type($3))
          err("invalid operands: arithmetic operation");
      }
      int t1 = get_type($1);
      code("\n\t\t%s\t", ar_instructions[$2 + (t1 - 1) * AROP_NUMBER]);

      gen_sym_name($1);
      code(",");
      gen_sym_name($3);
      code(",");

      free_if_reg($3);
      free_if_reg($1);

      $$ = take_reg();
      gen_sym_name($$);
      set_type($$, t1);

  ;
```

```
ADDS   -4(%14),-8(%14),%0
```

- generiši operaciju
- generiši ulazni operand 1
- generiši ulazni operand 2
- generiši izlazni operand

Preuzimanje povratne vrednosti – izraz (exp) u okviru dodele

```
a = f(a + b);
```

```
ADDS  -4(%14),-8(%14),%0        1
PUSH  %0                        2
CALL f                          3
ADDU %15,$4,%15                 4
MOV %13,%0                      5
MOV %0,-4(%14)                  6
```

Drugi MOV je dodela (u nastavku)

```
exp
  : literal
  | _ID
      {
        $$ = lookup_symbol($1, VAR|PAR);
        if($$ == NO_INDEX)
          err("'%s' undeclared", $1);
      }
  | function_call
  | _LPAREN num_exp _RPAREN
      { $$ = $2; }
  ;
```

```
MOV %13,%0

- generiši preuzimanje iz registra 13
```

```
exp
  : literal
  | _ID
      {
        $$ = lookup_symbol($1, VAR|PAR);
        if($$ == NO_INDEX)
          err("'%s' undeclared", $1);
      }
  | function_call
      {
        $$ = take_reg();
        gen_mov(FUN_REG, $$);
      }

  | _LPAREN num_exp _RPAREN
      { $$ = $2; }
  ;
```

```
MOV %13,%0

- generiši preuzimanje iz registra 13
```

Dodela

```
a = f(a + b);
```

```
ADDS  -4(%14),-8(%14),%0        1
PUSH  %0                        2
CALL f                          3
ADDU %15,$4,%15                 4
MOV %13,%0                      5
MOV %0,-4(%14)                  6
```

```
assignment_statement
  : _ID _ASSIGN num_exp _SEMICOLON
      {
        int idx = lookup_symbol($1, VAR|PAR);
        if(idx == NO_INDEX)
          err("invalid lvalue '%s' in assignment", $1);
        else
          if(get_type(idx) != get_type($3))
            err("incompatible types in assignment");
      }
  ;
```

```
MOV %0,-4(%14)

- generiši preuzimanje iz registra 13
```

```
assignment_statement
  : _ID _ASSIGN num_exp _SEMICOLON
      {
        int idx = lookup_symbol($1, VAR|PAR);
        if(idx == NO_INDEX)
          err("invalid lvalue '%s' in assignment", $1);
        else
          if(get_type(idx) != get_type($3))
            err("incompatible types in assignment");

        gen_mov($3, idx);
      }
  ;
```

```
MOV %0,-4(%14)

- generiši preuzimanje iz registra 13
```

Pojmovi bez GK:

program
function_list
variable_list
statement_list
statement
compound_statement

parametar
variable
literal